HOW QUANTUM COMPUTERS USE SHOR'S ALGORITHM TO CRACK RSA

- Shor's algorithm can break RSA and ECC, the public-key systems used to exchange keys and sign data.
- RSA's security rests on one assumption: factoring $N = p \times q$ is infeasible for a classical computer.
- The quantum part of Shor's algorithm only finds a period r. Every other step is ordinary arithmetic.



FOR EXAMPLE

Plain text is:`81, 0, 38, 71, 29, 31, 0, 82, 82, 32, 1`

Public key: (e,N)=(5,91)

Before we started what is e and N?

N: is the product of two primes p and q. In real RSA both are huge

e is public. To decrypt we still need the private exponent d, which requires knowing p and q.

WE WILL GET P AND Q USING PERIOD FINDING

$$f(x) = a^x \bmod N$$

1=gcd(a , N) in our example N= 91 then 1=gcd(a , 91) , a=2

$x) = 2^x \bmod 91$	x
57	9
23	10
46	11
1	12
2	13
4	14
8	15

$f(x) = 2^x \bmod 91$	x
2	1
4	2
8	3
16	4
32	5
64	6
37	7
74	8

As you can see, the values start to repeat

NOW WE GET R= 12

- P= $\gcd(2^{r/2}-1, n) = \gcd(2^6-1, 91) = 7$
- q= $\gcd(2^{r/2}+1, n) = \gcd(2^6+1, 91) = 13$

Euler's Totient Function

$$\phi(N) = (p-1)(q-1) = 72$$

Why we get euler totient function?

To get d(Private Exponent)

$$\gcd(e, \phi(N)) = 1$$

$$\gcd(e, 72) = 1$$

E=5

- Now we get d(Private Exponent)

$$e \cdot d \equiv 1 \pmod{\phi(N)}$$

$$5d \equiv 1 \pmod{72}$$

d=29

Private Key = (d , N) = (29 , 91)

Encryption = $m = c^d \bmod N$

$$c^{29} \bmod 91$$

الحرف	$c^{29} \bmod 91$	c Ciphertext	PNG
I	9	81	PNG
Space	0	0	PNG
L	12	38	PNG
O	15	71	PNG
V	22	29	PNG
E	5	31	PNG
Space	0	0	PNG
J	10	82	PNG
J	10	82	PNG
B	2	32	PNG
A	1	1	PNG

Encrypted text is = I LOVE JJBA

WHAT THIS EXAMPLE LEAVES OUT

- numbers: N = 91 is factored instantly by anyone. Breaking a 2048-bit N needs a large fault-tolerant quantum computer.
- Harvest now, decrypt later: data recorded today could be decrypted once such a machine exists.
- Defense: post-quantum algorithms standardized by NIST, such as ML-KEM and ML-DSA.

HOW QUANTUM COMPUTERS USE SHOR'S ALGORITHM TO CRACK ECC

ELLIPTIC CURVE CRYPTOGRAPHY

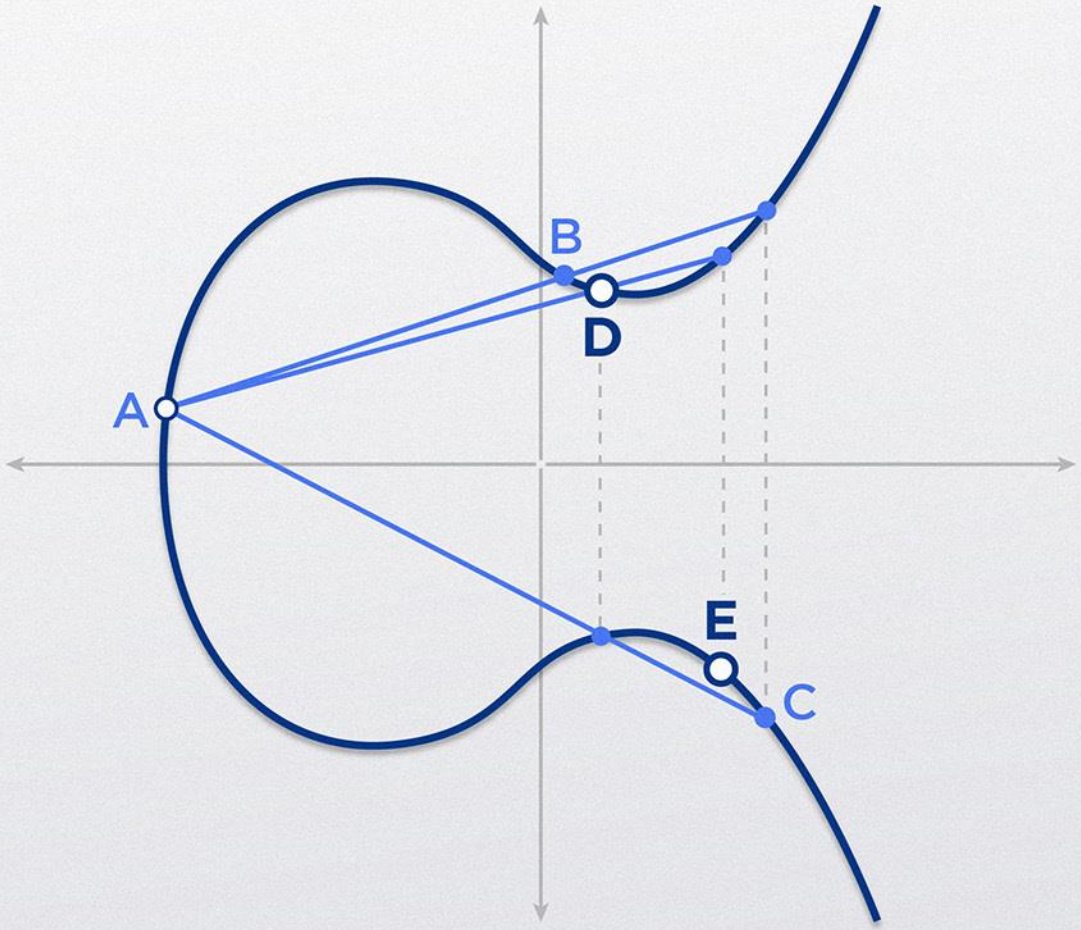


Figure: ECC

1. Starting at A
2. A dot B = -C (Draw a line from A to B and it intersects at -C)
3. Reflect across the X axis from -C to C
4. A dot C = -D (Draw a line from A to C and it intersects -D)
5. Reflect across the X axis from -D to D

ECC: THE SAME MESSAGE, A NEW KEY

- Goal: find the private key k and encrypted text.

i	1	2	3	4	5	6	7	8	9	10	11
C	(11,9)	(9,12)	(9,11)	(4,7)	(8,8)	(0,21)	(9,12)	(1,12)	(1,12)	(1,11)	(7,3)

Curve: $y^2 = x^3 + x + 4 \bmod 23$ Q(public key is (10 , 5)) & R(Ephemeral Public Point(13 ,12))

First, we find the Base Point by checking the points that satisfy the Elliptic Curve equation over 23 Then we select a suitable point as the Base Point(G)
First point(0 , 0) , last point (22 , 22) for Curve: $y^2 = x^3 + x + 4$
Now what point achieve $y^2 = x^3 + x + 4 \bmod 23$

G=(0, 2) (0, 21) (1, 11) (1, 12) (4, 7) (4, 16) (6, 7) (6, 16) (7, 7) (7, 16) (8, 10) (8, 13) (9, 11) (9, 12) (11, 9) (11, 14) (13, 3) (13, 20) (15, 0) (19, 6) (19, 17) (21, 9) (21, 14) (22, 9) (22, 14)

We need to check all these points until we find the suitable one

POINT DOUBLING AND ADDITION TO FIND CURVE ORDER (N)

Calculate the slope (s) of the tangent line at point P:

2P=(x3 , y3) $s = \frac{3x_1^2 + a}{2y_1} \pmod p$ $x_3 = s^2 - 2x_1 \pmod{23}$ $y_3 = s(x_1 - x_3) - y_1 \pmod{23}$

In our example g=(0,2) s=6 x3=13 y3=12 2G=(13 ,12)

3G = 2G + G $s = \frac{y_2 - y_1}{x_2 - x_1} \pmod p$ $x_3 = s^2 - x_1 - x_2 \pmod p$ $y_3 = s(x_1 - x_3) - y_1 \pmod p$

In our example g=(0,2) & 2G=(13,12) s=22 x3=11 y3=9 3G=(11 ,9)

1G: (0,2) PNG	**11G:** (10,5) PNG	**21G:** (14,18) PNG
2G: (13,12) PNG	**12G:** (17,9) PNG	**22G:** (15,17) PNG
3G: (11,9) PNG	**13G:** (8,15) PNG	**23G:** (9,12) PNG
4G: (1,12) PNG	**14G:** (18,9) PNG	**24G:** (7,3) PNG
5G: (7,20) PNG	**15G:** (18,14) PNG	**25G:** (1,11) PNG
6G: (9,11) PNG	**16G:** (8,8) PNG	**26G:** (11,14) PNG
7G: (15,6) PNG	**17G:** (17,14) PNG	**27G:** (13,11) PNG
8G: (14,5) PNG	**18G:** (10,18) PNG	**28G:** (0,21) PNG
9G: (4,7) PNG	**19G:** (22,18) PNG	**29G:** *O* PNG
10G: (22,5) PNG	**20G:** (4,16) PNG	-

- Public key(Q) = (10 , 5)= KG = k * (0 , 2) using **point multiplication** for k = 11

Ephemeral Public Point(R)=(13,12) , R=rG , 2G=(13 ,12) , r=2

Private Key S = kR = 11(13 , 12) = (15 , 17)

We need to subtract this point from each Ciphertext Point: M=C-S

In ECC, subtraction means adding the Negative Point.

-S=(15,-17 mod 23)=(15,6)

M=C+(15,6)

C_1=(11,9) , M= (11 , 9) + (15 , 6) M=(22,5)

In G table 10G=(22 ,5) M=10G m=10-1 = 9 IT will be letter I

i	Ciphertext C	بعد فك التشفير M	t (من جدول G)
PNG 1	PNG (11,9)	PNG (22,5)	PNG 10
PNG 2	PNG (9,12)	PNG (0,2)	PNG 1
PNG 3	PNG (9,11)	PNG (8,15)	PNG 13
PNG 4	PNG (4,7)	PNG (8,8)	PNG 16
PNG 5	PNG (8,8)	PNG (9,12)	PNG 23
PNG 6	PNG (0,21)	PNG (9,11)	PNG 6
PNG 7	PNG (9,12)	PNG (0,2)	PNG 1
PNG 8	PNG (1,12)	PNG (10,5)	PNG 11
PNG 9	PNG (1,12)	PNG (10,5)	PNG 11
PNG 10	PNG (1,11)	PNG (11,9)	PNG 3
PNG 11	PNG (7,3)	PNG (13,12)	PNG 2

Encrypted text is = I LOVE JJBA

RSA AND ECC UNDER SHOR

	RSA	ECC
Hard problem	Integer factoring	Elliptic-curve discrete log
Shor finds	Period r of a^x mod N	Hidden period of f(a, b) = a·G + b·Q
Result	p, q, then φ(N), then d	The private key k directly
Quantum cost	Modular exponentiation of large numbers	Many elliptic-curve point additions, but keys are much shorter
Defense	ML-KEM, ML-DSA	ML-KEM, ML-DSA